

WHAT LEAN CANNOT SAY: A MACHINE-CHECKED ANALYSIS OF WITTGENSTEIN'S *TRACTATUS*

ROBB WALTERS

Abstract. We present a machine-checked reconstruction of the semantic core of *Tractatus Logico-Philosophicus* in the LEAN proof assistant. The formalization models objects, states of affairs, worlds, and propositions, and proves truth-functional compositionality, functional completeness, and bivalence.

Beyond encoding, the system is parameterized over world models, allowing a precise separation between (i) structural invariants of the propositional syntax, (ii) model-dependent assumptions such as the independence of atomic facts, and (iii) formal limits of expressibility. We show that truth-functional compositionality holds in all models, while independence characterizes a specific “free” model and fails in constrained models.

To analyze the relation between object language and metalanguage, we introduce an **expresses** relation linking propositions to meta-level predicates. Using this, we prove that any proposition expressing a world-independent property must be a tautology or contradiction. Consequently, nontrivial propositions cannot express world-independent truths.

We further distinguish three notions of equivalence—syntactic identity, logical-form equivalence up to permutation, and semantic equivalence—and prove that the first strictly refines the other two, which are themselves provably incomparable: logical form neither determines nor is determined by truth conditions.

We formalize Wittgenstein's *N*-operator (joint denial) and settle TLP 5.52 in both directions: the quantifiers reduce to *N*-applications over any finite domain, while over an infinite domain no single *N*-application over finitely many instances expresses the universal quantifier—the Geach–Soames critique as a machine-checked theorem. We also prove full functional completeness (TLP 5.101): every truth-function on finitely many atoms is realized by some proposition.

We give an exact characterization of the sayable—over finitely many atoms, a world-property is expressible by a proposition iff it is invariant under pointwise- $\iff$ agreement of worlds—and show it fails over infinitely many atoms, where the totality property of TLP 1 (“the world is everything that is the case”) is invariant yet expressed by no proposition.

These results yield a formal decomposition of the *Tractatus* into invariants, assumptions, and limits, and provide a precise characterization of the saying/showing distinction as a theorem about expressibility in truth-functional languages. The formalization comprises 2,296 lines of LEAN 4, proves 80 formally verified results with 0 **sorry** obligations and 1 deliberate axiom (**axiom silence** : **True**, encoding TLP 7).¹

2020 *Mathematics Subject Classification*. Primary 03B70; Secondary 03A05, 03B35, 03B05.

Key words and phrases. Wittgenstein, *Tractatus Logico-Philosophicus*, proof assistant, Lean 4, formal verification, truth-functional semantics, saying/showing distinction, expressibility, *N*-operator.

§1. Introduction. Wittgenstein’s *Tractatus Logico-Philosophicus* [1] proposes that the world consists of atomic facts (Sachverhalte), that propositions are truth-functions of elementary propositions, and that the limits of language are the limits of the world. The text is terse, aphoristic, and deliberately resistant to formalization—yet its semantic core is a precise theory of truth-functional composition that invites formal reconstruction.

Several authors have pursued this invitation. Lokhorst [3] provides a set-theoretic reconstruction covering ontology, semantics, and propositional attitudes; Fogelin [4] initiates the debate over expressive completeness that Geach [8] and Soames [9] develop; Miller [10] develops a Tractarian semantics for predicate logic, responding in part to Fogelin and Carruthers [5]; Weiss [7] reconstructs the logical system with Π_1^1 -completeness results; Stokhof [6] traces the *Tractatus*’s influence on formal semantics. These reconstructions clarify the *Tractatus* as a semantic system but are carried out on paper, without machine verification. Separately, the proof-assistant community has formalized philosophical arguments [11, 12] and even complete metaphysical systems [13], but a Tractarian formalization has not appeared.

This paper contributes a machine-checked reconstruction in LEAN 4 [21] that goes beyond encoding to yield new analytical results. Our central claim is:

The saying/showing distinction of the Tractatus can be reformulated as a theorem: nontrivial propositions cannot express world-independent truths.

This is not a paraphrase of Wittgenstein but a provable consequence of the typed structure of the formalization. The key device is an **expresses** relation (Definition 5.1) that bridges the object language (propositions of type **Proposition** S) and the metalanguage (terms of type **Prop**), making the object/meta boundary visible at the type level.

More broadly, by parameterizing the formalization over a general **WorldModel** structure, we show that the claims of the *Tractatus* decompose into three classes:

1. **Structural invariants** that hold in every world model—compositionality, bivalence, De Morgan duality.
2. **Model-dependent assumptions** that hold in the unconstrained “free” model but fail in constrained models—principally, the independence of atomic facts.
3. **Formal limits** on expressibility—the collapse of world-independent content to tautology or contradiction, and the strict separation of syntactic, logical-form, and semantic equivalence.

The specific contributions of this paper are:

- (1) A parameterized world-model abstraction that cleanly separates structural invariants from modeling assumptions, enabling precise identification of which Tractarian claims are theorems of the syntax and which are properties of a particular world model (Section 2).
- (2) An **expresses** relation that makes the object/meta boundary a typed distinction, yielding the expressibility collapse theorem (Theorem 5.2): any proposition expressing a world-independent property is a tautology or contradiction (Section 5).

- (3) An equivalence analysis of three relations on propositions: syntactic identity strictly refines both logical-form equivalence and semantic equivalence, while the latter two are provably incomparable—logical form neither determines nor is determined by truth conditions. The permutation-based $\sim_{\text{form}}$ provides a formal candidate for Wittgenstein’s “logical form” (Section 5.4).
- (4) A formalization of the N -operator (joint denial) settling TLP 5.52 in both directions: proved for finite domains, and refuted—as a machine-checked theorem—for infinite ones, formalizing the Geach–Soames critique (Section 5.6).
- (5) Full functional completeness (TLP 5.101): every truth-function on finitely many atoms is realized by some proposition, with the inhabitation hypothesis this genuinely requires (Section 3.4).
- (6) An exact characterization of expressibility and the inexpressibility of totality: over finitely many atoms a world-property is expressible iff it is invariant under pointwise- $\iff$ agreement of worlds, whereas over infinitely many atoms the totality property of TLP 1 is invariant yet expressed by no proposition—a finite/infinite dichotomy turning on finite support (Section 5.3).
- (7) A three-way decomposition of all 80 formalized results into invariants, assumptions, and limits (Table 1, Appendix B).
- (8) A first-order extension using higher-order abstract syntax, proving compositionality lifts to quantified propositions (Section 5.5).

The formalization comprises 2,296 lines of LEAN 4 across six files.¹ All proofs compile with 0 `sorry` obligations. The sole axiom is `axiom silence : True` (TLP 7), a deliberate boundary marker discussed in Section 7.2.

The paper is organized as follows. Section 2 presents the formalization: types, evaluation, and the world-model abstraction. Section 3 establishes the structural invariants. Section 4 analyzes the model-dependent assumptions and exhibits countermodels. Section 5 proves the expressibility results and the separation of the three equivalence relations. Section 6 discusses related work. Section 7 reflects on the philosophical implications and limitations.

§2. The Formalization. We formalize the ontological skeleton of the *Tractatus* in LEAN 4. The design choices are deliberate modeling decisions, not exegetical claims about Wittgenstein’s intent. We state them explicitly so that the reader can assess the reconstruction’s scope and limitations.

Remark 2.1 (Choice of proof assistant). We use LEAN 4 [21] rather than Coq, Isabelle, or Agda for three reasons: (i) its dependent type theory allows definitions and proofs to coexist naturally, which suits a formalization that mixes data definitions with metatheorems; (ii) the Mathlib library provides automation (`simp`, `tauto`, `push_neg`) that keeps proofs concise; and (iii) LEAN’s tactic

¹`TractatusOntology.lean` (1,345 lines, 45 theorem/lemma declarations), `TractatusQuantifiers.lean` (284 lines, 9), `TractatusNOperator.lean` (239 lines, 11), `TractatusCompleteness.lean` (142 lines, 9), `TractatusExpressibility.lean` (138 lines, 3), and `TractatusDecidability.lean` (148 lines, 3), plus 1 deliberate axiom. Appendix B lists every declaration.

mode allows readable proof scripts that can be presented in a paper without excessive notation overhead.

Remark 2.2 (Classical logic). The formalization assumes classical logic throughout, via `Classical.em` and related principles imported from `Mathlib`. This is a deliberate choice matching the *Tractatus*’s bivalent semantics: every proposition is either true or false in every world (TLP 4.023). Theorems depending essentially on classical reasoning include bivalence, excluded middle as a tautology, double negation, the expressibility collapse theorem, and contingent-proposition variation. A constructive formalization would require restricting the world model or weakening several results.

2.1. Objects and States of Affairs. TLP 2.02: “The object is simple.” Objects are the substance of the world (TLP 2.021). We model them as an abstract type `TractObject` with no internal structure, capturing Wittgenstein’s insistence on simplicity.

A state of affairs (Sachverhalt) is a possible combination of objects (TLP 2.01). Rather than encoding the combinatorial structure of objects into states of affairs, we index states of affairs by an abstract type `S`:

```
variable (TractObject : Type)
variable (Sachverhalt : Type)
```

This choice leaves the internal structure of states of affairs unspecified—an interpretive decision that captures independence while remaining agnostic about combinatorial form (cf. TLP 2.0141). We note explicitly that `TractObject` is declared for conceptual completeness but plays no role in the subsequent formal development, which begins at the level of states of affairs; encoding the combination relation between objects and Sachverhalte is future work (Section 7.2).

2.2. Worlds. TLP 1: “The world is everything that is the case.” A world is determined by which states of affairs obtain. We model this as a predicate:

```
def World := Sachverhalt → Prop
```

Every function $S \rightarrow \text{Prop}$ counts as a world. This gives the full Boolean cube of truth-value assignments, making the independence of atomic facts (TLP 2.061) hold by construction. We return to this modeling choice and its consequences in Section 4.

2.3. Propositions. TLP 5: “The proposition is a truth-function of elementary propositions.” We define propositions as an inductive type built from elementary propositions, negation, and conjunction:

```
inductive Proposition (S : Type) where
| elementary : S → Proposition S
| neg        : Proposition S → Proposition S
| conj       : Proposition S → Proposition S → Proposition S
```

Disjunction, implication, biconditional, and NAND are defined as derived operations (TLP 5.101, 5.5), confirming that $\{\neg, \wedge\}$ is an adequate basis.

2.4. Semantic Evaluation. The core semantic function interprets propositions against a world:

```
def Proposition.eval (p : Proposition S) (w : World S) : Prop :=
  match p with
  | .elementary s => w s
  | .neg q       => ¬(q.eval w)
  | .conj q r    => q.eval w ∧ r.eval w
```

An elementary proposition `elementary s` is true in world w exactly when $w s$ holds. Negation and conjunction lift to their classical counterparts.²

2.5. Tautology, Contradiction, Nontriviality. A proposition is a *tautology* if it holds in every world, a *contradiction* if it holds in none, and *nontrivial* if it holds in some world and fails in another:

```
def IsTautology (p : Proposition S) : Prop :=
  ∀ w : World S, p.eval w
def IsContradiction (p : Proposition S) : Prop :=
  ∀ w : World S, ¬(p.eval w)
def Nontrivial (p : Proposition S) : Prop :=
  ∃ w1 w2 : World S, p.eval w1 ∧ ¬p.eval w2
```

We prove that nontriviality is equivalent to being neither a tautology nor a contradiction.³

2.6. General World Models. The definitions above hardwire the world model: every function $S \rightarrow \text{Prop}$ is a world. In many applications—physics, epistemic logic, constrained databases—not every assignment is legitimate. We therefore introduce a `WorldModel` structure that parameterizes the semantics:

```
structure WorldModel (S : Type) where
  W      : Type
  holds  : W → S → Prop
  nonempty : Nonempty W
```

The *free model* `freeModel` recovers the original semantics: $W = S \rightarrow \text{Prop}$ and `holds` is function application. We define a general evaluator `evalM` and prove that `evalM (freeModel S)` agrees with `Proposition.eval` (Theorem `evalM.free.eq.eval`).

This abstraction is the key structural device of the formalization. It allows us to state precisely which results depend on the choice of world model and which do not.

²The formalization also provides a `Bool`-valued evaluator `evalBool` for decidable computation and proves its correctness against `eval` (Theorem `evalBool.correct`). This engineering artifact enables `#eval` demonstrations but is not philosophically load-bearing.

³Theorem `nontrivial_iff_not_taut_not_contra` in `TractatusOntology.lean`.

§3. Structural Invariants. The following results hold in *every* world model—they are consequences of the inductive structure of propositions and classical logic alone, with no assumptions about which worlds exist.

3.1. Truth-Functional Compositionality.

THEOREM 3.1 (Compositionality, model-independent). *For any world model M , any proposition p , and any two worlds $w_1, w_2 \in M.W$: if $M.\text{holds } w_1 s \leftrightarrow M.\text{holds } w_2 s$ for every state of affairs s , then $\text{eval}_M p w_1 \leftrightarrow \text{eval}_M p w_2$.*

The proof is by structural induction on propositions. The elementary case follows from the hypothesis; negation and conjunction lift via **Iff.not** and **Iff.and**.

```

theorem truth_functional_compositionality_gen (p : Proposition S)
  (w1 w2 : M.W)
  (h : ∀ s : S, M.holds w1 s ↔ M.holds w2 s) :
  evalM M p w1 ↔ evalM M p w2 := by
  induction p with
  | elementary s => exact h s
  | neg q ih    => simp only [evalM, ih]
  | conj q r ihq ihr => simp only [evalM, ihq, ihr]

```

This establishes TLP 5 as a structural invariant: the truth-value of a compound proposition is determined by its elementary constituents regardless of which worlds the model admits.

3.2. Bivalence and Classical Tautologies. Elementary bivalence (TLP 4.023) is an instance of classical excluded middle:

THEOREM 3.2 (Bivalence). *For any state of affairs s and world w : $w s \vee \neg w s$.*

The standard tautologies follow: excluded middle ($p \vee \neg p$), double negation ($\neg \neg p \leftrightarrow p$), De Morgan’s laws, and modus ponens. We verify each as a formal theorem. All of these results depend on **Classical.em** (see Remark 2.2).

3.3. NAND Functional Completeness. TLP 5.5 anticipates the Sheffer stroke. We prove that negation and conjunction are both expressible via NAND:

THEOREM 3.3 (NAND completeness). *For any propositions p, q and world w :*
 1. $\text{nand}(p, p).\text{eval } w \leftrightarrow (\neg p).\text{eval } w$
 2. $(\neg \text{nand}(p, q)).\text{eval } w \leftrightarrow (p \wedge q).\text{eval } w$

Since $\{\neg, \wedge\}$ generates all truth-functions and NAND generates $\{\neg, \wedge\}$, NAND alone is functionally complete over the set of truth-functions expressible by the proposition type—confirming TLP 5.5.

3.4. Functional Completeness, Full Strength. The NAND result shows the interdefinability of the bases, but TLP 5.101 claims more: *every* truth-function of the elementary propositions arises from the propositional basis. We prove this in full strength (**TractatusCompleteness.lean**):

THEOREM 3.4 (Functional completeness, TLP 5.101). *Let S be finite, decidable, and nonempty. For every truth-function $g : (S \rightarrow \text{Bool}) \rightarrow \text{Bool}$ there is a proposition p with $p.\text{evalBool } w = g w$ for every assignment w .*

```

theorem functional_completeness {S : Type} [Fintype S] [DecidableEq S]
  [Nonempty S] (g : (S → Bool) → Bool) :
  ∃ p : Proposition S, ∀ w : S → Bool, p.evalBool w = g w

```

The construction is the disjunctive normal form: for each satisfying assignment v of g , a *minterm* conjoins, for every atom s , the literal `elementary s` or its negation according to v s ; the minterms of all satisfying assignments are then disjoined via the derived `disj`.

Remark 3.5 (`Nonempty S` is genuinely required). For empty S the type `Proposition S` is uninhabited (every proposition bottoms out in an elementary leaf), while $(S \rightarrow \text{Bool}) \rightarrow \text{Bool}$ still has two elements—so nothing realizes the constant functions. With at least one atom, the constants are realizable as $p \vee \neg p$ and $p \wedge \neg p$. Stating the theorem surfaced this hypothesis: a small instance of the paper’s running theme that formalization forces modeling commitments into the open.

§4. Model-Dependent Assumptions. Not every claim in the *Tractatus* is a structural invariant. The independence of atomic facts—perhaps the most famous ontological thesis of the text—is a property of a specific world model.

4.1. Independence in the Free Model. We formalize independence as a typeclass:

```

class IndependentWorlds (S : Type) where
  realizable : ∀ assignment : S → Prop,
    ∃ w : World S, ∀ s, w s ↔ assignment s

```

The free model trivially satisfies this: every assignment *is* a world.

```

instance : IndependentWorlds S :=
  ⟨fun a => ⟨a, fun _ => Iff.rfl⟩⟩

```

Independence is thus not a theorem about logic but a consequence of the modeling choice `World S = S → Prop`. This distinction—between what is proved and what is assumed by the encoding—is precisely the kind of clarification that a typed formalization forces.

4.2. Failure in Constrained Models. To show that independence is genuinely contingent on the model, we exhibit two countermodels.

Abstract countermodel. Consider two states of affairs a, b with $a \neq b$, where b must co-occur with a . Define `ConstrainedWorld S a b = {w : S → Prop | w a → w b}`. Then:

THEOREM 4.1 (Independence fails in constrained models). *If $a \neq b$, then there is no constrained world realizing the assignment $\{a \mapsto \top, b \mapsto \perp\}$.*

```

theorem constrained_independence_fails (a b : S) (hab : a ≠ b) :
  ¬ ∀ assignment : S → Prop,
    ∃ cw : ConstrainedWorld S a b,

```

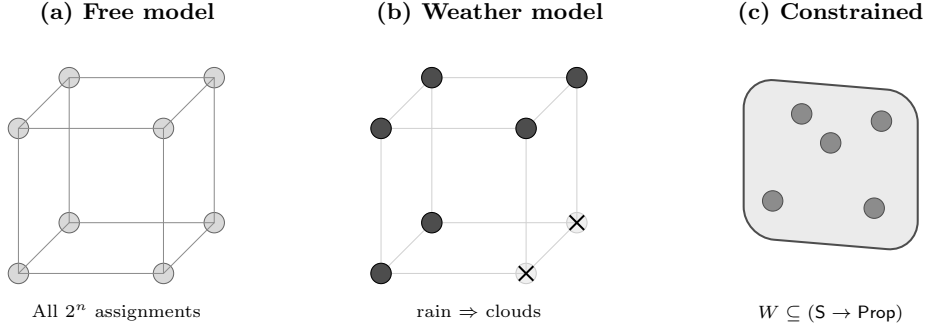


FIGURE 1. World-model architecture. (a) The free model admits all 2^n truth-value assignments as worlds. (b) The weather model excludes assignments violating $\text{rain} \Rightarrow \text{clouds}$ (crossed vertices). (c) An abstract constrained model selects an arbitrary subset of the Boolean cube. Compositionality holds in all three; independence holds only in (a).

```

  ∀ s, cw.val s ↔ assignment s := by
intro h
let bad : S → Prop := fun s => s = a
obtain ⟨w, hw⟩, hmatch := h bad
have ha : w a := (hmatch a).mpr rfl
have hb : w b := hw ha
exact hab ((hmatch b).mp hb).symm

```

Weather model. A more concrete example: three states of affairs (rain, clouds, snow) with the physical constraint *rain implies clouds*. The weather model restricts worlds to those satisfying this constraint.

THEOREM 4.2 (Weather model). *In the weather model, there is no world where rain holds but clouds does not.*

These results make explicit a point that is often asserted informally: TLP 2.061 (“Atomic facts are independent of one another”) is not a logical law but a constraint on the space of possible worlds. Our formalization turns this observation into a proof.

Notably, Wittgenstein himself later retreated from the independence thesis. In “Some Remarks on Logical Form” [2], he acknowledged that the color-exclusion problem—the mutual incompatibility of determinate color predicates (e.g., a point cannot be both red and blue)—demonstrates that certain atomic facts are not logically independent. Our constrained models formalize precisely this kind of failure: the weather model’s constraint that rain implies clouds is structurally analogous to the color-exclusion problem. In both cases, physical or conceptual constraints on states of affairs reduce the space of possible worlds below the full Boolean cube.

Figure 1 illustrates the relationship between the free model and constrained models.

§5. Formal Limits: Expressibility and Equivalence. The deepest results of the formalization concern the boundaries of what the object language can express. These are not claims within the Tractarian system but *about* it— theorems at the meta-level that formalize Wittgenstein’s saying/showing distinction.

5.1. The expresses Relation. We introduce a typed bridge between the object language and the metalanguage:

DEFINITION 5.1 (**expresses**). A proposition $p : \text{Proposition } S$ *expresses* a meta-level property $P : \text{Prop}$ if and only if p ’s truth value tracks P in every world:

$$\text{expresses } p \ P \stackrel{\text{def}}{=} \forall w : \text{World } S, \ p.\text{eval } w \leftrightarrow P$$

```
def expresses (p : Proposition S) (P : Prop) : Prop :=
  ∀ w : World S, p.eval w ↔ P
```

The type signature makes the object/meta distinction visible: p lives in the object language, P lives in LEAN’s metalanguage, and **expresses** is the claim that they correspond. This is the formalization’s key analytical device.

5.2. Expressibility Collapse.

THEOREM 5.2 (Expressibility collapse). *If a proposition p expresses a world-independent property $P : \text{Prop}$, then p is a tautology or a contradiction.*

PROOF. Suppose **expresses** $p \ P$ holds. Then for any two worlds w_1, w_2 :

$$p.\text{eval } w_1 \leftrightarrow P \leftrightarrow p.\text{eval } w_2$$

so p has the same truth value in all worlds. By excluded middle on $p.\text{eval } w_0$ for an arbitrary world w_0 , either p holds everywhere (tautology) or fails everywhere (contradiction). $\neg$

```
theorem saying_showing_triviality (q : Proposition S)
  (P : Prop)
  (h : expresses q P) :
  IsTautology q ∨ IsContradiction q := by
  apply world_constant_taut_or_contra
  intro w₁ w₂
  exact (h w₁).trans (h w₂).symm
```

The contrapositive is immediate and philosophically sharper:

COROLLARY 5.3 (Saying requires world-dependence). *If p is nontrivial (i.e., true in some world and false in another), then there is no $P : \text{Prop}$ such that **expresses** $p \ P$.*

```
theorem nontrivial_cannot_express_world_independent
  (p : Proposition S) (P : Prop)
  (hnt : Nontrivial p) :
  ¬ expresses p P := by
```

```

intro h
exact nontrivial_expressibility_requires_world_dependence
p P hnt h

```

This is the paper’s headline result. It says: if a proposition genuinely “says” something—i.e., carves out a nontrivial region of logical space—then what it says cannot be a world-independent truth. Genuine content requires variation across possible worlds. What is world-independent can only be “shown” by the structure of the formalism itself, not “said” within it.

This transforms the saying/showing distinction from a philosophical thesis into a provable constraint on truth-functional languages.

Remark 5.4 (On the definitional character of the collapse). A natural objection is that the expressibility collapse theorem is *trivial*: since $P : \mathbf{Prop}$ is world-independent by construction (it is a closed term in LEAN’s metalanguage), the collapse to tautology or contradiction follows almost definitionally from the definition of **expresses**.

We acknowledge this. The proof is indeed short—two lines of tactic code. The contribution lies not in the proof’s difficulty but in three aspects of the *formulation*:

(a) *Making the object/meta boundary typed.* The definition of **expresses** forces the distinction between $p : \mathbf{Proposition\ S}$ (object language) and $P : \mathbf{Prop}$ (metalanguage) to be a type-level constraint. This makes the boundary machine-checkable rather than merely conventional.

(b) *The analogy to other definitional results in logic.* Many foundational results in logic are “definitional” in a similar sense. Tarski’s undefinability theorem rests on the diagonal lemma, which is itself a consequence of how truth predicates are defined. The incompleteness theorems depend on the coding of syntax. In each case, the contribution is the *conceptual apparatus* that makes the result statable, not the complexity of the derivation. Our **expresses** relation plays an analogous role: it provides the vocabulary in which the saying/showing distinction becomes a theorem.

(c) *The constraint is genuinely informative.* The collapse theorem rules out a precise class of claims: no nontrivial proposition can serve as a “name” for a fixed metalinguistic truth. This is not obvious from the definition of **expresses** alone—one must also establish (via **world_constant_taut_or_contra**) that world-invariant truth values force triviality.

Remark 5.5 (The **Nonempty S** assumption). Exactly one theorem in the development requires **[Nonempty S]** (i.e., that the type of states of affairs is inhabited): **structEq_ne_semEq**, which must exhibit a concrete elementary proposition and therefore needs at least one atom. (The separation witnesses of Theorem 5.11 take their two distinct atoms as explicit hypotheses $a \neq b$ rather than assuming inhabitation.) The expressibility collapse (Theorem 5.2) and its corollaries need no such assumption: a witness world always exists, because **World S** = $\mathbf{S} \rightarrow \mathbf{Prop}$ is inhabited by the constant predicate $\lambda _.$ True even when **S** is empty. (For empty **S** the collapse is vacuous for a different reason: **Proposition S** has no inhabitants, since every proposition bottoms out in an elementary leaf.) An earlier version

of the formalization carried `[Nonempty S]` on the collapse chain; removing it is a small but genuine strengthening, and it locates the inhabitation requirement exactly where it belongs: in the witnesses, which must name atoms, not in the expressibility limits, which only quantify over worlds.

5.3. What Can Be Said, Exactly. The collapse theorem says what a proposition *cannot* express: a fixed world-independent truth $P : \text{Prop}$. Its positive complement asks what a proposition *can* express when the meta-level content is allowed to depend on the world. Say a property $P : \text{World } S \rightarrow \text{Prop}$ is *invariant* if worlds agreeing pointwise up to $\iff$ agree on P : for all w_1, w_2 , if $\forall s. w_1 s \iff w_2 s$ then $P w_1 \iff P w_2$. This is precisely the truth-functional condition of Section 3.1, now imposed on a metalinguistic predicate rather than derived for a proposition. Over finitely many atoms, invariance is exactly expressibility (`TractatusExpressibility.lean`):

THEOREM 5.6 (Exact characterization of the expressible). *Let S be finite, decidable, and nonempty. A world-property $P : \text{World } S \rightarrow \text{Prop}$ is expressible by some proposition if and only if P is invariant under pointwise- $\iff$ agreement of worlds.*

```
theorem expressible_iff_iff_invariant {S : Type} [Fintype S]
  [DecidableEq S] [Nonempty S] (P : World S → Prop) :
  (∃ p : Proposition S, ∀ w : World S, p.eval w ↔ P w) ↔
  (∀ w₁ w₂ : World S, (∀ s, w₁ s ↔ w₂ s) → (P w₁ ↔ P w₂))
```

The forward direction is truth-functional compositionality (Theorem 3.1): any property a proposition expresses must respect pointwise- $\iff$ agreement, because the proposition sees a world only through the truth-values of its elementaries. The backward direction is functional completeness (Theorem 3.4): an invariant P descends to a Boolean truth-function on assignments $S \rightarrow \text{Bool}$, which some proposition realizes; `evalBool.correct` transfers the realization back to the `Prop`-valued semantics. The saying/ showing distinction thus has a sharp two-sided boundary over finite atoms: the sayable is exactly the truth-functionally invariant.

The finiteness hypothesis is not incidental. A proposition has only finitely many atoms, and its truth-value depends only on them—a *finite-support* property. Let $p.\text{atoms}$ collect the elementaries occurring in p .

LEMMA 5.7 (Finite support). *For any proposition p and worlds w_1, w_2 , if $w_1 s \iff w_2 s$ for every $s \in p.\text{atoms}$, then $p.\text{eval } w_1 \iff p.\text{eval } w_2$.*

Over an infinite atom type, finite support defeats the very first proposition of the *Tractatus*. TLP 1—“the world is everything that is the case”—renders as the totality property $\lambda w. \forall s. w s$. This property is invariant under pointwise- $\iff$ agreement, so by Theorem 5.6 it would be expressible over any finite S . Yet over an infinite S it is not:

THEOREM 5.8 (Inexpressibility of totality, TLP 1). *If S is infinite, no proposition expresses the totality property: there is no p with $p.\text{eval } w \iff (\forall s. w s)$ for all worlds w .*

```
theorem totality_not_expressible {S : Type} [Infinite S] :
  ¬ ∃ p : Proposition S, ∀ w : World S, p.eval w ↔ (∀ s, w s)
```

The proof compares the all-true world with the world true exactly on $p.\text{atoms}$. The two agree on every atom of p , so by finite support (Lemma 5.7) p takes the same value at both. But totality holds at the first and—because an infinite type has an atom outside the finite list $p.\text{atoms}$ —fails at the second. So no p tracks totality. The three new results carry the same axiom footprint as the rest of the Aristotle-assisted development (`propext`, `Classical.choice`, `Quot.sound`).

Remark 5.9 (The finite/infinite dichotomy). Theorems 5.6 and 5.8 together draw a sharp line. For *finitely* many atoms, invariance and expressibility coincide: everything truth-functionally invariant is sayable. For *infinitely* many atoms the two come apart, and TLP 1 itself witnesses the gap—an invariant property that no proposition expresses.

The reading this invites should be stated carefully, without overclaiming. The theorem is not a formal vindication of TLP 7 or of the ladder metaphor (Section 7.2), and it does not show that “the world” is unsayable *simpliciter*. It is a result about *this* object language over an infinite atom type: within the finitary truth-functional syntax the *Tractatus* makes available, the totality of facts is not the content of any proposition, even though it is a perfectly well-defined predicate in the metalanguage that states the theorem. The unsayability is relative to the expressive means of the object language, and it is the metalanguage—not silence—that says it. What the result does supply is a precise sense in which a Tractarian language over infinitely many atoms cannot say, from inside, that a given world is the whole of what is the case: the gesture at totality lands outside the reach of the proposition.

5.4. Three Equivalence Relations. The formalization distinguishes three notions of proposition equivalence:

DEFINITION 5.10 (Three equivalence relations). Let $p, q : \text{Proposition } S$.

1. **Structural equivalence** ($p \sim_{\text{struct}} q$): p and q are built from identical constructors in the same tree shape with identical atoms.
2. **Logical-form equivalence** ($p \sim_{\text{form}} q$): there exists a bijective renaming of atoms (a permutation $\sigma : S \xrightarrow{\sim} S$) such that $p.\text{rename } \sigma = q$.
3. **Semantic equivalence** ($p \sim_{\text{sem}} q$): p and q have the same truth value in every world: $\forall w, p.\text{eval } w \leftrightarrow q.\text{eval } w$.

We prove that $\sim_{\text{form}}$ is an equivalence relation (reflexive by the identity permutation, symmetric by inverse, transitive by composition). How do the three relations relate? They do *not* form a chain. Structural equivalence strictly refines both coarser relations, but logical-form equivalence and semantic equivalence are *incomparable*: neither contains the other.

THEOREM 5.11 (Separation). *As relations on Proposition S (for S with at least two atoms):*

1. $\sim_{\text{struct}} \subsetneq \sim_{\text{form}}$ and $\sim_{\text{struct}} \subsetneq \sim_{\text{sem}}$;
2. $\sim_{\text{form}} \not\subseteq \sim_{\text{sem}}$ and $\sim_{\text{sem}} \not\subseteq \sim_{\text{form}}$;

3. $\sim_{\text{struct}} \subsetneq \sim_{\text{form}} \cap \sim_{\text{sem}}$.

PROOF. (1) *Refinement*. Structural equality implies propositional equality (by induction), which implies logical-form equivalence via the identity permutation (`structEq_implies_formEq`) and semantic equivalence trivially (`structEq_implies_semEq`).

(2) *Incomparability*. For any two distinct atoms $a \neq b$, `elementary(a)` and `elementary(b)` are $\sim_{\text{form}}$ (via the swap permutation) but not $\sim_{\text{sem}}$: they differ in truth value at the world where exactly a obtains (`formEq_not_semEq_witness`). Conversely, for any atom s , $\neg\neg(\text{elementary } s)$ is $\sim_{\text{sem}}$ to `elementary s` (double negation) but not $\sim_{\text{form}}$, since renaming preserves tree depth (`semEq_not_formEq_witness`). The same two pairs witness strictness in (1): the first separates $\sim_{\text{struct}}$ from $\sim_{\text{form}}$, the second separates $\sim_{\text{struct}}$ from $\sim_{\text{sem}}$.

(3) *The intersection*. For distinct atoms $a \neq b$, the commuted conjunctions $a \wedge b$ and $b \wedge a$ share their logical form (swap permutation) and their truth conditions (commutativity of $\wedge$) yet are not structurally equal (`formEq_semEq_not_structEq_witness`). So even the common refinement $\sim_{\text{form}} \cap \sim_{\text{sem}}$ is strictly coarser than $\sim_{\text{struct}}$.

All witnesses are stated in the formalization for an arbitrary atom type S , taking the required distinct atoms as explicit hypotheses; the running example (`rain`, `snow` in `TwoFacts`) is recovered as an instance, checked by `example` declarations. The theorem as stated is therefore machine-checked at full generality. $\dashv$

Remark 5.12 (Decidability). Over finite atom types the three relations are not merely separated but *decidable*: the module `TractatusDecidability` proves `semEq_iff_evalBool`—semantic equivalence coincides with agreement of the computable `Bool`-valued evaluators on all `Bool` assignments, for an arbitrary atom type—and derives decision procedures `decideSemEq` and `decideFormEq` once S is a `Fintype`. So the separation of Theorem 5.11 is checkable by pure computation: at `TwoFacts` the witnesses are decided by `#eval`, mirroring the evaluator demonstrations of Section 2.4.

Remark 5.13 (On truth-table isomorphism). What survives of the intuition that logical form is “truth conditions up to relabeling”? The formalization proves `formEq_implies_truth_table_iso`: if $p \sim_{\text{form}} q$, there is a world-relabeling f with $p.\text{eval } w \leftrightarrow q.\text{eval } (f \ w)$ for all w . But this relation is far weaker than it looks, and the formalization proves that too (`truth_table_iso_of_nontrivial`): with classical choice, *any* proposition is truth-table-isomorphic to *any* nontrivial proposition—relabel each world to a satisfying or refuting world of q according to p ’s truth value there. So truth-table isomorphism cannot serve as an intermediate notion between $\sim_{\text{form}}$ and $\sim_{\text{sem}}$; we record it as a lemma, not as a rung of a hierarchy.

Figure 2 illustrates the relationships. This clarifies a long-standing ambiguity in the *Tractatus*. Wittgenstein’s notion of “logical form” (TLP 4.0141) is neither syntactic identity nor truth-conditional equivalence. Our $\sim_{\text{form}}$ —defined via bijective atom renaming—provides a formal candidate, and the separation

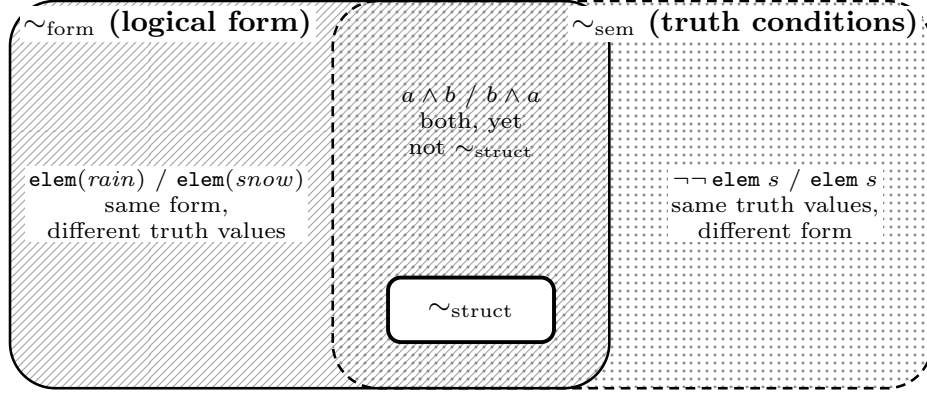


FIGURE 2. The three equivalence relations. $\sim_{\text{form}}$ and $\sim_{\text{sem}}$ are incomparable—each region contains a machine-checked witness—and $\sim_{\text{struct}}$ sits strictly inside their intersection (Theorem 5.11).

theorem shows it is genuinely independent of truth conditions: form neither determines nor is determined by semantic content. The two dimensions meet only above $\sim_{\text{struct}}$, and even their intersection does not collapse to syntax (part 3 of Theorem 5.11).

Remark 5.14 (Relation to Tarskian invariance). Characterizing a notion by invariance under permutation recalls Tarski’s criterion of logicity [18]: a notion is logical iff it is invariant under all permutations of the domain of individuals. Our $\sim_{\text{form}}$ permutes *atoms* to characterize shared *form between propositions*, rather than permuting individuals to characterize *logical constants*; the two invariance criteria operate at different levels. The separation theorem adds something the Tarskian tradition does not address: form-invariance and truth-conditional identity cut across each other. There is also a structural echo of that tradition’s central difficulty: just as permutation invariance is charged with *overgenerating* logical constants [16, 17], our `truth_table_iso_of_nontrivial` (Remark 5.13) shows that world-relabeling overgenerates equivalences—which is precisely why we do not admit it as a rung between $\sim_{\text{form}}$ and $\sim_{\text{sem}}$. Spinney [19] gives a recent treatment of logical form and logical space in the *Tractatus* on which our formal candidate can be brought to bear.

5.5. First-Order Extension. In a companion file (`TractatusQuantifiers.lean`), we extend the propositional calculus with first-order quantifiers using higher-order abstract syntax (HOAS):

```
inductive FOProp (S : Type) (D : Type) : Type where
| lift      : Proposition S → FOProp S D
| negFO     : FOProp S D → FOProp S D
| conjFO    : FOProp S D → FOProp S D → FOProp S D
| forall_   : (D → FOProp S D) → FOProp S D
| exists_   : (D → FOProp S D) → FOProp S D
```

The HOAS encoding leverages LEAN’s metalanguage for variable binding, avoiding explicit substitution machinery. This substantially reduces proof overhead but ties the formalization more tightly to LEAN’s metatheory than a de Bruijn index encoding would—in particular, induction over HOAS terms requires care, since the recursor for HOAS inductives must track the functional argument.

We prove that compositionality lifts to the first-order level:

THEOREM 5.15 (First-order compositionality). *For any $p : \text{FOProp } S \ D$ and worlds w_1, w_2 agreeing on all states of affairs, $p.\text{eval } w_1 \leftrightarrow p.\text{eval } w_2$.*

We also establish eight additional results: universal–existential duality (TLP 5.52), universal distribution over conjunction, existential distribution over disjunction (reverse direction), lift preservation, vacuous quantifier elimination (requiring `Nonempty D`), double negation at the first-order level, De Morgan duality for quantifiers, and first-order bivalence.

The results above are standard consequences of the semantic definitions. The philosophically loaded question is whether Wittgenstein’s N -operator can ground quantification (TLP 5.52)—for infinite domains, Geach and Soames [8, 9] argue it cannot, since the N -operator is a finitary syntactic operation. We now settle both directions of that question.

5.6. The N -Operator: Finite Success, Infinite Failure. We formalize the N -operator as joint denial over a nonempty argument list (TLP 5.502), in `TractatusNOperator.lean`:

```
def NOp (p : Proposition S) (ps : List (Proposition S)) : Proposition
  S :=
  ps.foldl1 (fun acc q => Proposition.conj acc (Proposition.neg q))
    (Proposition.neg p)
```

Its defining semantics is joint denial: $\text{NOp } p \ ps$ is true exactly when every argument is false (`NOp.eval`); a first-order analogue `NOpFO` behaves identically. Three consequences confirm TLP 5.5/5.512: binary N is NOR, unary N is negation, and conjunction is $N(N(p), N(q))$ —the single operation suffices for the propositional basis.

For quantifiers, TLP 5.52 holds *exactly* on finite domains:

THEOREM 5.16 (TLP 5.52, finite domains). *Over a completely enumerated domain $d :: ds$ (every $x : D$ occurs in the list), for every family $f : D \rightarrow \text{FOProp } S \ D$ and world w : $\forall x. fx$ has the same truth value as $N(\neg fd, \neg fd_1, \dots)$, and $\exists x. fx$ the same as $\neg N(fd, fd_1, \dots)$ (*forall-as-NOpFO*, *exists-as-NOpFO*).*

Conversely, the Geach–Soames critique is itself a theorem:

THEOREM 5.17 (Infinite-domain obstruction). *Let D be infinite and take atoms $S = D$. For every choice of finitely many instances $d :: ds$, the N -application $N(\neg \text{elementary } d, \neg \text{elementary } d_1, \dots)$ differs in truth value from $\forall x. \text{elementary } x$ at some world (*no-finite-NOp-for-forall*).*

PROOF SKETCH. Evaluate both sides at the world $w := (\cdot \in d :: ds)$. The N -application is true there, since every listed instance obtains; the universal

quantifier is false, since an infinite domain has an element outside any finite list. $\neg$

This sharpens the dialectic of the expressive-completeness debate: what Geach and Soames urged informally is, in this reconstruction, a short countermodel. What the theorem does *not* settle is Wittgenstein’s own intent—whether TLP 5.52’s “all values” was ever meant as a finitary list.

Iterated N : TLP 6. The results above concern a *single* N -application. TLP 6 makes a stronger claim: every proposition arises from the elementary propositions by *successive* applications of N . We formalize the closure of the elementaries under N as an inductive predicate **NGen**—the base case is any elementary proposition, and the step case closes under **NOp** p ps for N -generated p and ps —and prove that this closure exhausts the propositional fragment up to semantic equivalence.

THEOREM 5.18 (N -generation completeness, TLP 6). *For every proposition p there is a proposition q with **NGen** q and $q \sim_{\text{sem}} p$: every proposition is semantically equivalent to one built from elementary propositions by iterated N -application (**nGen-complete**).*

PROOF SKETCH. Induct on p . An elementary proposition is N -generated outright. For $\neg p$, take $N(q_p)$ where q_p generates p ; unary N is negation (**neg.via.NOp**), and semantic equivalence is a congruence for $\neg$ (**semEq_neg_congr**). For $p \wedge r$, take $N(N(q_p), N(q_r))$; this is conjunction (**conj.via.NOp**), and semantic equivalence is a congruence for $\wedge$ (**semEq_conj_congr**). $\neg$

Combined with functional completeness (Theorem 3.4), the reach of iterated joint denial is now exact: every truth-function over finitely many atoms is realized by some proposition, and that proposition is in turn semantically equivalent to one generated from the elementaries by successive N . This is TLP 6’s “successive applications” claim for the propositional fragment, up to semantic equivalence. What remains open is the form-series *notation* $[\bar{p}, \bar{\xi}, N(\bar{\xi})]$ of TLP 6 itself and its infinitary reading (see Section 7.2).

§6. Related Work.

Formal reconstructions of the *Tractatus*. Lokhorst [3] provides the most comprehensive prior formal reconstruction, covering ontology, semantics, and propositional attitudes in a set-theoretic framework. His axiomatization of independence and his truth-functionality theorem (every sentence is a truth-function of elementary sentences) overlap with our encoding, but he does not formalize the saying/showing boundary itself. Weiss [7] gives a 50-page reconstruction of the *Tractatus*’s logical system, analyzing the N -operator and form-series in detail and proving that tautology-hood is Π_1^1 -complete for countable domains—a striking complexity result that sharpens the Geach–Soames critique. Miller [10] develops a Tractarian semantics for predicate logic that is independent of the metaphysics of logical atomism, engaging with both Fogelin’s [4] critique of expressive completeness and Carruthers’ [5] book-length treatment of Tractarian

semantics. Stokhof [6] reads the *Tractatus* as a proto-formal semantics, tracing its influence through Carnap’s state descriptions to possible-worlds semantics, and argues that the saying/showing distinction arises from the universalism of the framework—a philosophical diagnosis our expressibility collapse theorem makes precise. Spinney [19] examines the relation between logical form and logical space in the *Tractatus*; our permutation-based $\sim_{\text{form}}$ (Section 5.4) offers a formal candidate for the notion that discussion targets. All of these works provide formal clarity but are carried out on paper; none offers machine-checked proofs, and none parameterizes over world models to isolate which claims are structural invariants versus modeling assumptions.

The expressive completeness debate. Fogelin [4] initiates the modern debate over whether the *Tractatus*’s logical resources are expressively complete. Geach [8] argues that the N -operator cannot ground quantification over infinite domains, since it is a finitary operation. Soames [9] sharpens this: the *Tractatus*’s claim that all of logic reduces to truth-functional operations on elementary propositions is undermined when the domain of quantification is infinite. Carruthers [5] provides a sympathetic reconstruction of Tractarian semantics that attempts to address some of these concerns. Most recently, Lampert and Nakano [15] argue that the classical undecidability proofs do not refute Wittgenstein’s early conception of logic on its own terms—he rejects principles those proofs assume—and supply a purely logical refutation that avoids such principles. Our formalization is neutral on this decidability dispute: we formalize the semantic core (worlds, truth-functions, expressibility), not the *ab*-notation decision procedure whose fate that debate concerns. Our formalization engages the critique directly (Section 5.6): TLP 5.52 is proved for finite domains, and the infinite-domain failure is itself a machine-checked theorem (`no_finite_NOp_for_forall`)—to our knowledge the first formal rendering of the Geach–Soames objection.

Proof assistants and philosophy. Benz Müller and colleagues [11] have used Isabelle to formalize ontological arguments and modal logic. Their approach uses shallow semantic embeddings of higher-order modal logic into Isabelle’s meta-logic, which contrasts with our direct encoding in dependent type theory: we define the Tractarian object language explicitly as an inductive type, whereas shallow embeddings reuse the proof assistant’s logic as the object logic. Fitelson and Zalta [12] pioneer “computational metaphysics,” using automated reasoners to verify theorems in abstract-object theory. Scherf [13] provides the most directly comparable work: a machine-checked axiomatization of Advaita Vedānta in LEAN 4 (69 axioms, 40+ theorems), demonstrating that complete philosophical systems can be formalized in dependent type theory. Corfield [20] makes a sustained case for homotopy type theory as a foundation for philosophical reasoning, complementing the formal-verification approach taken here. To our knowledge, no prior work formalizes any part of the *Tractatus* in any proof assistant, nor uses the type system to isolate the saying/showing boundary as a provable constraint.

§7. Discussion.

7.1. What the Formalization Shows. The formalization yields three kinds of insight.

First, it makes the *Tractatus*’s implicit modeling decisions explicit. The choice $\text{World } S = S \rightarrow \text{Prop}$ is not neutral: it builds independence into the foundations. By parameterizing over `WorldModel`, we separate what follows from the propositional syntax (compositionality) from what follows from the choice of world space (independence). This distinction is available in principle but rarely made precise in the philosophical literature.

Second, it transforms the saying/showing distinction from a metaphilosophical thesis into a theorem with a two-line proof. The `expresses` relation is the key: once the object/meta boundary is a type distinction, the collapse result (Theorem 5.2) follows by transitivity of biconditional. See Remark 5.4 for a discussion of the definitional character of this result and why the formulation is nonetheless a genuine contribution.

Third, the separation theorem (Theorem 5.11) provides a precise formal candidate for Wittgenstein’s notion of “logical form.” The proved incomparability of $\sim_{\text{form}}$ and $\sim_{\text{sem}}$ shows that logical form, understood as connective structure up to atom permutation, is not reducible to truth conditions—and, conversely, that truth conditions do not determine form. The two are orthogonal dimensions of a proposition’s identity, meeting only above syntactic identity.

Remark 7.1 (On automated proof search). The N -operator and functional-completeness proofs were found by an automated prover from our theorem statements (see the Acknowledgments). The division of labor is itself a datum for this paper’s theme: proof search automates what can be *said*—derivations of stated propositions—while the choice of definitions (`expresses`, `formEq`, `NOp`, the world-model parameterization) is where the philosophical content lives and resists delegation. Statement-level errors are precisely the kind proof search cannot catch: a prover asked to prove a false generalization simply fails, while a reviewer can refute it.

7.2. Limitations. Several aspects of the *Tractatus* lie beyond the scope of this formalization.

Object structure. The type `S` is abstract: we do not model how objects combine into states of affairs. TLP 2.0141 (“The possibility of its occurrence in atomic facts is the form of the object”) suggests a richer dependent-type encoding where the form of an object constrains its combinatorial possibilities. This would make independence a substantive theorem rather than a definitional consequence.

The form-series. The N -operator is formalized (Section 5.6), and Wittgenstein’s claim that all propositions arise by *successive* applications of N now holds for the propositional fragment up to semantic equivalence: every proposition is semantically equivalent to one generated from the elementaries by iterated N (Theorem 5.18). What remains open is the form-series *notation* of TLP 6 itself, $[\bar{p}, \bar{\xi}, N(\bar{\xi})]$, together with its infinitary reading—the recursive schema in which the same operation is applied to its own results without bound. Our `NGen` closure is finitary and unstructured (a mere inductive predicate), not Wittgenstein’s indexed general form of the proposition.

The picture theory. The *Tractatus*’s claim that propositions are *pictures* of reality (TLP 2.1–2.174) involves a notion of structural isomorphism between language

and world that goes beyond truth-functional evaluation. Our formalization captures the truth-conditional aspect but not the pictorial-isomorphism aspect. Self-reference and the ladder. TLP 6.54 famously declares that the propositions of the *Tractatus* itself are “nonsensical” and must be “thrown away” after use. Our formalization is, by design, a metalanguage that talks *about* the Tractarian object language. It cannot formalize its own status as a ladder to be discarded—this is the point where type theory, like any formal system, reaches its own saying/showing boundary.

We mark this boundary with a deliberate axiom:

`axiom silence : True`

This is not a missing proof but a design boundary. The axiom states nothing contentful (**True** is already provable); it marks the point where the formalization acknowledges its own limits.

Remark 7.2 (Why True). The choice of **True** as the axiom’s type is deliberate. Because **True** is already provable in LEAN (via **trivial**), the axiom **silence** adds no logical content whatsoever—it introduces no new assumptions and changes no proof obligations. Its content is its *label*, not its type. The name “silence” marks a boundary; the type **True** ensures that boundary is inert. This mirrors TLP 7’s status as a proposition that, by Wittgenstein’s own account, says nothing: it is a gesture toward what lies beyond the expressible, not an assertion within it.

7.3. Beyond the Tractatus. The expressibility collapse theorem (Theorem 5.2) is formulated in terms of Tractarian propositions, but it depends only on two structural features: (i) the object language is truth-functional (proposition truth values are determined by evaluation against worlds), and (ii) the metalanguage is typed, so that a “world-independent property” is simply a term of type **Prop** with no free world variable. Any truth-functional object language embedded in a typed metalanguage with this structure will exhibit the same collapse.

This means the result is not specific to the *Tractatus*. Classical propositional logic, Boolean algebras interpreted over sets of valuations, and modal propositional logics (when the accessibility relation is removed) all satisfy the preconditions. The collapse theorem is a general fact about the typed embedding of truth-functional languages: the type system enforces a gap between world-dependent content (nontrivial propositions) and world-independent truth (metalinguistic facts), and no nontrivial proposition can bridge it.

For logicians who may not share an interest in Wittgenstein exegesis, this generality is the main takeaway: the saying/showing distinction, stripped of its philosophical mystique, is a typing constraint that any truth-functional formalization must respect.

7.4. Future Work. Several directions for further investigation arise naturally. First, with iterated N -application now settled up to semantic equivalence (Theorem 5.18), the remaining form-series work is to formalize TLP 6’s general propositional form $[\bar{p}, \bar{\xi}, N(\bar{\xi})]$ as an indexed recursive schema—and to give

its infinitary reading a coherent semantics—rather than the finitary, unstructured *NGen* closure we prove complete here. Second, encoding object structure via dependent types—so that states of affairs are typed combinations of objects constrained by form (TLP 2.0141)—would make independence a substantive theorem rather than a definitional consequence of the world type. Third, extending the *expresses* relation to world-dependent properties ($P : \text{World } S \rightarrow \text{Prop}$) would allow a finer analysis of what nontrivial propositions *can* express, complementing the current result about what they cannot. Fourth, formalizing the doxastic extension of Tractarian semantics along the lines of Lokhorst [3] would test whether the invariant/assumption/limit decomposition extends to propositional attitudes. Fifth, a companion development by the author already generalizes the constrained countermodels of Section 4 to arbitrary lists of Horn-clause constraints (and biconditional constraints via symmetric closure), and organizes world models under a refinement preorder with the free model as maximum; integrating that material would deepen the assumption tier of the decomposition.

§8. Conclusion. We have presented a machine-checked reconstruction of the semantic core of *Tractatus Logico-Philosophicus* in LEAN 4. The formalization proves 80 formally verified results across six files, decomposes the *Tractatus* into structural invariants, model-dependent assumptions, and formal limits of expressibility, and characterizes the saying/showing distinction as a theorem: nontrivial propositions cannot express world-independent truths. It settles TLP 5.52 in both directions—the *N*-operator grounds quantification over finite domains and provably fails to over infinite ones—and proves TLP 5.101 in full strength. It also fixes the sayable exactly: over finitely many atoms a world-property is expressible iff it is invariant under pointwise- $\iff$ agreement of worlds, while over infinitely many atoms the totality property of TLP 1 is invariant yet expressed by no proposition—so the totality of facts, sayable in the metalanguage, lies outside the reach of the object language whose propositions have finite support.

The equivalence analysis separates three relations on propositions: syntactic identity strictly refines logical-form equivalence and semantic equivalence, which are themselves provably incomparable. Logical form is thus not a waystation between syntax and truth conditions but an independent dimension of a proposition’s identity—a reading of TLP 4.0141 that only becomes visible under formalization.

These results suggest that proof assistants are a productive tool for philosophical analysis—not merely for verifying known results, but for revealing structural constraints that are invisible when the analysis is conducted on paper.

The formalization is publicly available at <https://github.com/rjwalters/tractatus>, with a browsable proof gallery at <https://leangenius.org/proof/tractatus-ontology>.

Acknowledgments. The proofs in `TractatusNOperator.lean`, `TractatusCompleteness.lean`, `TractatusExpressibility.lean`, and `TractatusDecidability.lean` were produced by Harmonic’s Aristotle automated theorem prover [14] from theorem statements authored by us (submitted with sorry placeholders; statements

unchanged), and were re-verified by the Lean kernel against the repository’s pinned toolchain. Their axiom footprint is `propext`, `Classical.choice`, `Quot.sound` only. For `TractatusExpressibility.lean` and `TractatusDecidability.lean`, the support the Aristotle problem files inlined (re-declared syntax and semantics, the `Bool`-valued evaluator, truth-functional compositionality, and the disjunctive-normal-form construction) is de-duplicated against the built development; only the headline statements—`expressible_iff_iff_invariant`, `eval_depends_only_on_atoms`, `totality_not_expressible` (expressibility) and `semEq_iff_evalBool`, `decideSemEq`, `decideFormEq` (decidability)—together with a derived `DecidableEq` instance and their finite-support scaffolding remain. All definitions, theorem statements, and the remaining proofs are ours.

Appendix A. Build Instructions. The formalization is built with Lean 4 using the lake build system. To reproduce:

1. Install Lean 4 via elan:


```
curl -sSf https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh | sh
```
2. Clone the repository:


```
git clone https://github.com/rjwalters/tractatus.git
```
3. The `lean-toolchain` file pins the version to `leanprover/lean4:v4.26.0`.
4. The `lakefile.lean` declares a dependency on `Mathlib` (the formalization uses `import Mathlib.Tactic`).
5. Build: `lake build`

The build produces zero errors, zero warnings, and zero sorry obligations. A GitHub Actions workflow in the repository runs `lake build` on every push, so the build status of the public sources is continuously verified. The sole axiom (`axiom silence : True`) is intentional and discussed in Section 7.2.

Proof files:

- `proofs/TractatusOntology.lean` — 1,345 lines, 45 declarations
- `proofs/TractatusQuantifiers.lean` — 284 lines, 9 declarations
- `proofs/TractatusNOperator.lean` — 239 lines, 11 declarations
- `proofs/TractatusCompleteness.lean` — 142 lines, 9 declarations
- `proofs/TractatusExpressibility.lean` — 138 lines, 3 declarations
- `proofs/TractatusDecidability.lean` — 148 lines, 3 declarations

Appendix B. Complete Theorem Listing. Tables 1 and 2 classify every theorem and lemma declaration in the formalization. The “Class” column indicates whether the result is a structural Invariant, a model-dependent Assumption or counterexample, or a formal Limit on expressibility. Results marked H are helper lemmas for the equivalence apparatus (renaming and `formEq`). Results marked E are engineering artifacts (`Bool`-valued evaluation). All Lean names are given in full; they can be located in the source files by exact text search.

TABLE 1. Complete listing of all 80 theorem/lemma declarations plus 1 axiom, classified by the three-way decomposition (Part 1 of 2: *TractatusOntology.lean*). **I** = invariant, **A** = assumption/counterexample, **L** = limit, **H** = equivalence helper, **E** = engineering.

#	Lean name	TLP	File	Cl.
<i>TractatusOntology.lean</i> (45 declarations)				
1	evalBool_correct	5.1	Ont.	E
2	truth_functional_compositionality_gen	5	Ont.	I
3	evalM_free_eq_eval	—	Ont.	I
4	tautology_is_world_invariant	6.1	Ont.	I
5	contradiction_holds_nowhere	4.46	Ont.	I
6	elem_prop_bivalence	4.023	Ont.	I
7	truth_functional_compositionality	5	Ont.	I
8	elementary_independence	2.061	Ont.	A
9	double_negation	5.101	Ont.	I
10	de_morgan_disj	5.101	Ont.	I
11	de_morgan_conj	5.101	Ont.	I
12	excluded_middle_tautology	4.46	Ont.	I
13	conj_neg_contradiction	4.46	Ont.	I
14	impl_semantics	5.101	Ont.	I
15	biimp_semantics	5.101	Ont.	I
16	modus_ponens	4.121	Ont.	I
17	nand_expresses_neg	5.5	Ont.	I
18	nand_expresses_conj	5.5	Ont.	I
19	constrained_independence_fails	2.061	Ont.	A
20	weather_independence_fails	2.061	Ont.	A
21	rename_id	4.014	Ont.	H
22	rename_comp	4.014	Ont.	H
23	rename_eval	4.014	Ont.	H
24	formEq_refl	4.0141	Ont.	H
25	formEq_symm	4.0141	Ont.	H
26	formEq_trans	4.0141	Ont.	H
27	eq_of_structEq (private)	4.0141	Ont.	H
28	structEq_implies_formEq	4.0141	Ont.	L
29	structEq_implies_semEq	4.0141	Ont.	L
30	formEq_implies_truth_table_iso	4.0141	Ont.	L
31	truth_table_iso_id_implies_semEq	4.0141	Ont.	L
32	truth_table_iso_of_nontrivial	4.0141	Ont.	L
33	formEq_not_structEq_witness	4.0141	Ont.	L
34	semEq_not_formEq_witness	4.0141	Ont.	L
35	formEq_not_semEq_witness	4.0141	Ont.	L
36	formEq_semEq_not_structEq_witness	4.0141	Ont.	L
37	structEq_ne_semEq	4.0141	Ont.	L
38	world_constant_taut_or_contra	4.46	Ont.	L
39	saying_showing_triviality	4.12	Ont.	L
40	contingent_propositions_vary	4.46	Ont.	L
41	nontrivial_of_not_taut_not_contra	4.46	Ont.	L
42	nontrivial_iff_not_taut_not_contra	4.46	Ont.	L
43	proposition_seven	7	Ont.	L
44	nontrivial_expressibility_requires_world_dependence	4.12	Ont.	L
45	nontrivial_cannot_express_world_independent	4.12	Ont.	L

TABLE 2. Theorem listing, Part 2 of 2: the remaining five files and the axiom (class codes as in Table 1).

#	Lean name	TLP	File	Cl.
<i>TractatusQuantifiers.lean</i> (9 declarations)				
46	truth_functional_compositionality_fo	5	Quant.	I
47	forall_exists_duality	5.52	Quant.	I
48	forall_distrib_conj	5.52	Quant.	I
49	exists_distrib_disj_reverse	5.52	Quant.	I
50	lift_eval	—	Quant.	I
51	forall_vacuous	—	Quant.	I
52	double_negation_fo	5.101	Quant.	I
53	not_exists_forall_not	5.52	Quant.	I
54	fo_bivalence	4.023	Quant.	I
<i>TractatusNOperator.lean</i> (11 declarations)				
55	NOp_eval	5.502	NOp.	I
56	NOpF0_eval	5.502	NOp.	I
57	NOp_pair_eval	5.5	NOp.	I
58	neg_via_NOp	5.512	NOp.	I
59	conj_via_NOp	5.5	NOp.	I
60	forall_as_NOpF0	5.52	NOp.	I
61	exists_as_NOpF0	5.52	NOp.	I
62	no_finite_NOp_for_forall	5.52	NOp.	L
63	semEq_neg_congr	6	NOp.	H
64	semEq_conj_congr	6	NOp.	H
65	nGen_complete	6	NOp.	I
<i>TractatusCompleteness.lean</i> (9 declarations)				
66	disj_evalBool	5.101	Comp.	E
67	literal_evalBool	5.101	Comp.	E
68	truePropOf_evalBool	5.101	Comp.	E
69	falsePropOf_evalBool	5.101	Comp.	E
70	conjFold_evalBool	5.101	Comp.	E
71	minterm_evalBool	5.101	Comp.	E
72	bigDisj_evalBool	5.101	Comp.	E
73	dnf_evalBool	5.101	Comp.	E
74	functional_completeness	5.101	Comp.	I
<i>TractatusExpressibility.lean</i> (3 declarations)				
75	expressible_iff_iff_invariant	5	Expr.	L
76	eval_depends_only_on_atoms	5	Expr.	L
77	totality_not_expressible	1	Expr.	L
<i>TractatusDecidability.lean</i> (3 declarations)				
78	semEq_iff_evalBool	4.0141	Dec.	E
79	decideSemEq	4.0141	Dec.	E
80	decideFormEq	4.0141	Dec.	E
<i>Axiom</i>				
—	axiom_silence : True	7	Ont.	L

REFERENCES

- [1] L. Wittgenstein, *Tractatus Logico-Philosophicus*, Kegan Paul, 1921. Translated by C. K. Ogden; reprinted by Routledge, 2001.
- [2] L. Wittgenstein, "Some remarks on logical form," *Proceedings of the Aristotelian Society, Supplementary Volumes*, vol. 9, pp. 162–171, 1929.
- [3] G.-J. C. Lokhorst, "Ontology, semantics and philosophy of mind in Wittgenstein's *Tractatus*: A formal reconstruction," *Erkenntnis*, vol. 29, no. 1, pp. 35–75, 1988.
doi: 10.1007/BF00166365.
- [4] R. J. Fogelin, *Wittgenstein*, 2nd ed., London: Routledge, 1987. (First edition 1976.)
- [5] P. Carruthers, *Tractarian Semantics: Finding Sense in Wittgenstein's Tractatus*, Oxford: Blackwell, 1989.
- [6] M. Stokhof, "The architecture of meaning: Wittgenstein's *Tractatus* and formal semantics," in *Wittgenstein's Enduring Arguments*, D. Levy and E. Zamuner, Eds. London: Routledge, 2008, pp. 211–244.
- [7] M. Weiss, "Logic in the *Tractatus*," *The Review of Symbolic Logic*, vol. 10, no. 1, pp. 1–50, 2017.
doi: 10.1017/S1755020316000472.
- [8] P. T. Geach, "Wittgenstein's operator N ," *Analysis*, vol. 41, no. 4, pp. 168–171, 1981.
doi: 10.1093/analys/41.4.168.
- [9] S. Soames, "Generality, truth functions, and expressive capacity in the *Tractatus*," *The Philosophical Review*, vol. 92, no. 4, pp. 573–589, 1983.
doi: 10.2307/2184881.
- [10] H. Miller, "Tractarian semantics for predicate logic," *History and Philosophy of Logic*, vol. 16, no. 2, pp. 197–215, 1995.
doi: 10.1080/01445349508837249.
- [11] C. Benz Müller and B. Woltzenlogel Paleo, "An object-logic explanation of the inconsistency in Gödel's ontological theory," *Journal of Applied Logic*, vol. 24, pp. 253–265, 2017.
doi: 10.1016/j.jal.2017.01.001.
- [12] B. Fitelson and E. N. Zalta, "Steps toward a computational metaphysics," *Journal of Philosophical Logic*, vol. 36, no. 2, pp. 227–247, 2007.
doi: 10.1007/s10992-006-9038-7.
- [13] M. Scherf, "The formal logic of Advaita Vedānta: A complete axiomatization and consistency proof," Unpublished manuscript, 2025. Original repository github.com/matthew-scherf/Advaita (since removed); archived at <https://web.archive.org/web/20260219072022/https://github.com/matthew-scherf/advaita>.
- [14] Harmonic, *Aristotle* (automated theorem prover for Lean 4), 2026. Available: <https://aristotle.harmonic.fun>. Accessed 2026-07-12.
- [15] T. Lampert and A. Nakano, "A logical refutation of Wittgenstein's early philosophy of logic," *History and Philosophy of Logic*, pp. 1–19, 2025.
doi: 10.1080/01445340.2025.2498308.
- [16] G. Sher, *The Bounds of Logic: A Generalized Viewpoint*, MIT Press, 1991.
- [17] D. Bonnay, "Logicity and invariance," *Bulletin of Symbolic Logic*, vol. 14, no. 1, pp. 29–68, 2008.
doi: 10.2178/bsl/1208358843.
- [18] A. Tarski, "What are logical notions?" (ed. J. Corcoran), *History and Philosophy of Logic*, vol. 7, no. 2, pp. 143–154, 1986.
doi: 10.1080/01445348608837096.
- [19] O. T. Spinney, "Logical form and logical space in Wittgenstein's *Tractatus*," *Synthese*, vol. 200, article 16, 2022.
doi: 10.1007/s11229-022-03470-y.

[20] D. Corfield, *Modal Homotopy Type Theory: The Prospect of a New Logic for Philosophy*, Oxford University Press, 2020.

doi: 10.1093/oso/9780198853404.001.0001.

[21] L. de Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” in *Proceedings of CADE-28*, Springer LNCS, 2021.

doi: 10.1007/978-3-030-79876-5_37.

INTEGRATED PLASMONICS CORPORATION
SAN FRANCISCO, CA, USA

E-mail: robb@integratedplasmonics.com